

CS5340 - Uncertainty Modelling in AI

Lecture 5: Factor Graph and the Junction Tree Algorithm

Scribed by: Claude (Anthropic) | Curated By: Assoc. Prof. Lee Gim Hee
Semester 1, AY2026/27 — National University of Singapore

Disclaimer: This scribe was generated by **Claude (Anthropic)** and curated by **Assoc. Prof. Lee Gim Hee**. It is a **supplementary revision aid** that summarizes the lecture slides, which remain the **primary learning material** prepared by the lecturer. Despite curation, the scribe may still contain errors, omissions, or simplifications. Please use it only alongside the official slides, textbook references, and your own lecture notes.

1 Lecture Overview

- **Motivation:** Lecture 4 gave us variable elimination (one query at a time, any graph) and sum-product (all marginals, but only on trees). Three gaps remain: (i) many useful models are **not** trees but contain only **local** cycles; (ii) we often want the **single most probable configuration**, not marginals; (iii) we want all marginals on **general loopy** graphs without re-running elimination per query.
- **Goal:** close these gaps with three tools – **factor graphs**, the **max-product** algorithm, and the **junction tree** algorithm.
- **Roadmap:**
 1. Represent a joint distribution as a **factor graph** and convert DGMs/UGMs into factor graphs.
 2. Run **sum-product on factor trees** with variable-to-factor (ν) and factor-to-variable (μ) messages.
 3. Formulate the **MAP** problem and solve it with **MAP-elimination** and the **max-product** algorithm (with back-tracking for configurations).
 4. Introduce **cluster graphs**, the **running intersection property** and **junction trees**.
 5. Run **sum-product on a junction tree**, and **construct** a junction tree from a DGM/UGM via triangulation and a maximum spanning tree.

Learning Outcomes

By the end of this lecture, you should be able to:

1. Represent a joint distribution with a factor graph, and use it to compute the marginal/conditional probabilities.
2. Use the max-product algorithm to find the maximal probability and its configurations.
3. Convert a DGM/UGM into the junction tree and use it to compute the marginal/conditional probabilities.

2 Factor Graphs

DGMs and UGMs both express a global function of several variables as a **product of factors** over subsets of those variables. **Factor graphs** make this decomposition **explicit** by adding extra nodes for the factors themselves. Unlike DGMs and UGMs, factor graphs are **not** designed to display

conditional independence; their purpose is to show the **precise form of the factorization**.

Definition : Factor Graph

A factor graph is a **bipartite graph** $\mathcal{G}(\mathcal{V}, \mathcal{F}, \mathcal{E})$, where

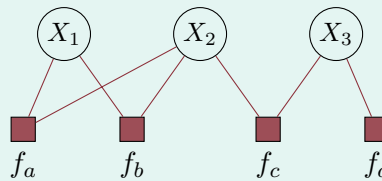
- vertices $\mathcal{V} \in \{X_1, \dots, X_n\}$ index the **random variables** (drawn as **round** nodes),
- vertices $\mathcal{F} \in \{\dots, f_s, \dots\}$ index the **factors** (drawn as **square** nodes), and
- undirected edges $\mathcal{E}$ link each factor node f_s to **all** variable nodes X_s that f_s depends on.

The joint distribution is written as a product of factors

$$p(\mathbf{x}) = \prod_s f_s(\mathbf{x}_s),$$

where $\mathbf{x}_s$ denotes a subset of the variables $\{X_1, \dots, X_n\}$ and each factor f_s is a function of its corresponding subset.

Worked Example: A Factor Graph Is More Specific Than a UGM



$$p(\mathbf{x}) = f_a(x_1, x_2) f_b(x_1, x_2) f_c(x_2, x_3) f_d(x_3).$$

- $f_a(x_1, x_2)$ and $f_b(x_1, x_2)$ are defined over the **same set of variables**. In a UGM their product would simply be **lumped together** into one clique potential.
- Similarly, $f_c(x_2, x_3)$ and $f_d(x_3)$ could be combined into a single potential over X_2 and X_3 .
- The factor graph **keeps such factors explicit**, so it conveys more detailed information about the underlying factorization.

(Image source: C. Bishop, "Pattern Recognition and Machine Learning".)

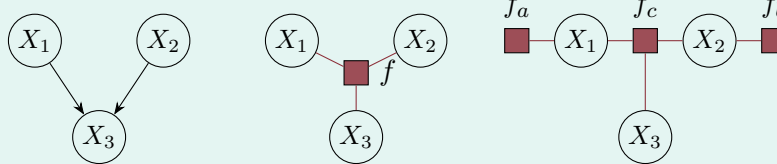
2.1 Converting DGMs and UGMs into Factor Graphs

Key Rule: DGM/UGM $\rightarrow$ Factor Graph

- **DGM:** the factorization is $p(x_1, \dots, x_N) = \prod_{i=1}^N p(x_i | x_{\pi_i})$. Represent each **local conditional distribution** $p(x_i | x_{\pi_i})$ as a factor $f_s(\mathbf{x}_s)$.
- **UGM:** the factorization is $p(\mathbf{y} | \boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} \prod_{c \in \mathcal{C}} \psi_c(\mathbf{y}_c | \boldsymbol{\theta}_c)$. Represent each **potential over a maximal clique** as a factor $f_s(\mathbf{x}_s)$. The normalizing coefficient $1/Z$ can be viewed as a factor defined over the **empty set** of variables.
- There may be **several different factor graphs** for the same DGM/UGM; the factor graph lets us be **more specific** about the precise form of the factorization.

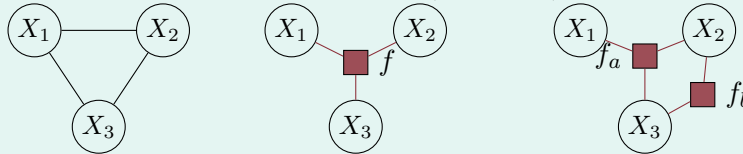
Worked Example: Two Factor Graphs for the Same Distribution

Directed graph $p(x_1)p(x_2)p(x_3 | x_1, x_2)$:



Middle: $f(x_1, x_2, x_3) = p(x_1)p(x_2)p(x_3 | x_1, x_2)$. Right: $f_a(x_1) = p(x_1)$, $f_b(x_2) = p(x_2)$, $f_c(x_1, x_2, x_3) = p(x_3 | x_1, x_2)$.

Undirected graph with single clique potential $\psi(x_1, x_2, x_3)$:

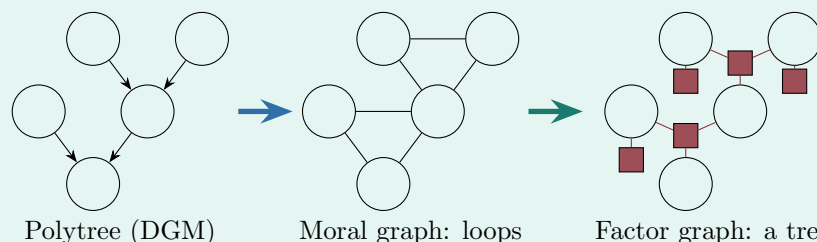


Middle: $f(x_1, x_2, x_3) = \psi(x_1, x_2, x_3)$. Right: $f_a(x_1, x_2, x_3) f_b(x_2, x_3) = \psi(x_1, x_2, x_3)$. (Image source: C. Bishop.)

Key Rule: Factor Graphs Can Remove Local Cycles

Factor graphs give an **alternative representation** for the sum-product algorithm on “tree-like” graphs. More importantly, some DGMs/UGMs with **local cycles become trees** when converted to factor graphs: in both examples above, the triangle X_1, X_2, X_3 (the moral graph of the v-structure, or the UGM clique) becomes a **star-shaped factor tree** around the single factor $f(x_1, x_2, x_3)$.

Worked Example: Polytrees



- **Cycles appear** after converting the directed polytree to an undirected (moral) graph.
- The **local cycles disappear** after factor graph conversion, since each family $p(x_i | x_{\pi_i})$ becomes one factor node.
- The factor graph conversion can be done **directly from the DGM**, skipping moralization.

(Image source: C. Bishop.)

2.2 Sum-Product on Factor Trees

Goal: compute **all singleton marginal probabilities** under the factorized representation of the joint. As in Lecture 4, we pass messages – but now there are **two kinds**:

1. Messages ν : flow **from variable to factor** nodes.

2. Messages μ : flow **from factor to variable** nodes.

Definition : Neighborhood Sets

- $\mathcal{N}(s) \subset \mathcal{V}$: the neighbors of a factor node $s \in \mathcal{F}$, i.e. the indices of **all variables referenced by f_s** .
- $\mathcal{N}(i) \subset \mathcal{F}$: the neighbors of a variable node $i \in \mathcal{V}$, i.e. the set of **all factors that reference X_i** .

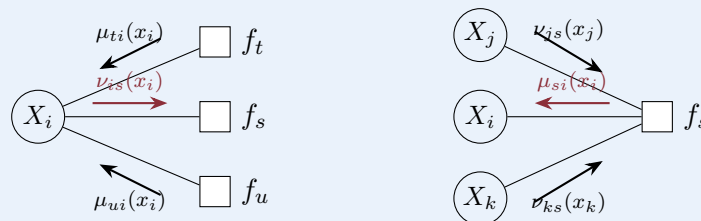
Definition : Variable-to-Factor and Factor-to-Variable Messages

Variable $\rightarrow$ factor. The message from X_i to f_s is the product of all incoming messages to X_i **other than** the one from f_s :

$$\nu_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$$

Factor $\rightarrow$ variable. The message from f_s to X_i multiplies the factor by all incoming messages to f_s **other than** the one from X_i , then sums out every variable except x_i :

$$\mu_{si}(x_i) = \sum_{x_{\mathcal{N}(s) \setminus i}} \left(f_s(x_{\mathcal{N}(s)}) \prod_{j \in \mathcal{N}(s) \setminus i} \nu_{js}(x_j) \right)$$



(Image source: M. I. Jordan, 2002.)

Key Rule: Messages from Leaf Nodes and the Message-Passing Protocol

- A **leaf variable node** sends the unit message $\nu_{is}(x_i) = 1$ (empty product).
- A **leaf factor node** (which depends on x_i only) sends $\mu_{si}(x_i) = f_s(x_i)$ (nothing to sum out).
- **Protocol:** a node can send a message to a neighboring node **when (and only when)** it has received messages from **all of its other neighbors**. This applies to **both** variable and factor nodes.

Key Rule: Marginal Probability of a Node

Once X_i has received messages from all its neighbors,

$$p(x_i) \propto \prod_{s \in \mathcal{N}(i)} \mu_{si}(x_i) = \nu_{is}(x_i) \mu_{si}(x_i) \quad \text{for any } s \in \mathcal{N}(i),$$

since $\nu_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$ already contains every incoming message except μ_{si} .

Algorithm: Factor Tree Sum-Product (Jordan, 2002)

```

procedure SUM-PRODUCT( $\mathcal{T}, E$ )                                ▷ main steps
  EVIDENCE( $E$ )                                                    ▷ 1
   $f \leftarrow \text{CHOOSEROOT}(\mathcal{V})$ 
  for  $s \in \mathcal{N}(f)$  do
     $\mu\text{-COLLECT}(f, s)$                                           ▷ 2
  end for
  for  $s \in \mathcal{N}(f)$  do
     $\nu\text{-DISTRIBUTE}(f, s)$                                        ▷ 3
  end for
  for  $i \in \mathcal{V}$  do
    COMPUTEMARGINAL( $i$ )                                           ▷ 4
  end for
end procedure

procedure EVIDENCE( $E$ )                                          ▷ 1: conditioning  $\rightarrow$  marginalization
  for  $i \in E$  do
     $\psi^E(x_i) = \psi(x_i) \delta(x_i, \bar{x}_i)$ 
  end for
  for  $i \notin E$  do
     $\psi^E(x_i) = \psi(x_i)$ 
  end for
end procedure

procedure  $\mu\text{-COLLECT}(i, s)$                                     ▷ 2: collect from leaves to root
  for  $j \in \mathcal{N}(s) \setminus i$  do
     $\nu\text{-COLLECT}(s, j)$ 
  end for
   $\mu\text{-SENDMESSAGE}(s, i)$                                           ▷  $\mu_{si}(x_i)$ 
end procedure
procedure  $\nu\text{-COLLECT}(s, i)$ 
  for  $t \in \mathcal{N}(i) \setminus s$  do
     $\mu\text{-COLLECT}(i, t)$ 
  end for
   $\nu\text{-SENDMESSAGE}(i, s)$                                           ▷  $\nu_{is}(x_i)$ 
end procedure

procedure  $\nu\text{-DISTRIBUTE}(i, s)$                                 ▷ 3: distribute from root to leaves
   $\nu\text{-SENDMESSAGE}(i, s)$ 
  for  $j \in \mathcal{N}(s) \setminus i$  do
     $\mu\text{-DISTRIBUTE}(s, j)$ 
  end for
end procedure
procedure  $\mu\text{-DISTRIBUTE}(s, i)$ 
   $\mu\text{-SENDMESSAGE}(s, i)$ 
  for  $t \in \mathcal{N}(i) \setminus s$  do

```

```

     $\nu$ -DISTRIBUTE( $i, t$ )
  end for
end procedure

```

```

procedure COMPUTEMARGINAL( $i$ )

```

▷ 4: marginal probability

```

     $p(x_i) \propto \nu_{is}(x_i) \mu_{si}(x_i)$ 

```

```

end procedure

```

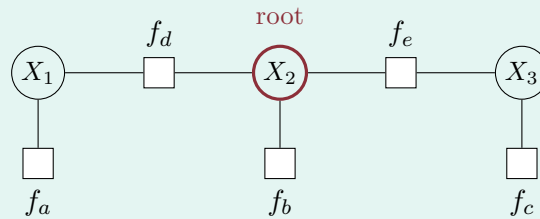
Evidence enters through singleton factor nodes $f(x_i) = \psi^E(x_i)$ attached to each variable.

Worked Example: Sum-Product on a Three-Node Factor Chain

The UGM chain $X_1 - X_2 - X_3$ with

$$p(x \mid \bar{x}_E) = \frac{1}{Z^E} \psi^E(x_1) \psi^E(x_2) \psi^E(x_3) \psi(x_1, x_2) \psi(x_2, x_3)$$

is converted into the factor tree below, with $f_d = \psi(x_1, x_2)$, $f_e = \psi(x_2, x_3)$, $f_a = \psi^E(x_1)$, $f_b = \psi^E(x_2)$, $f_c = \psi^E(x_3)$. We choose X_2 as the root.



Collect (inward):

- Leaf factors: $\mu_{a1}(x_1) = \psi^E(x_1)$, $\mu_{b2}(x_2) = \psi^E(x_2)$, $\mu_{c3}(x_3) = \psi^E(x_3)$.
- Variable $\rightarrow$ factor: $\nu_{1d}(x_1) = \mu_{a1}(x_1)$, $\nu_{3e}(x_3) = \mu_{c3}(x_3)$.
- Factor $\rightarrow$ variable: $\mu_{d2}(x_2) = \sum_{x_1} \psi(x_1, x_2) \mu_{a1}(x_1)$, $\mu_{e2}(x_2) = \sum_{x_3} \psi(x_2, x_3) \mu_{c3}(x_3)$.

Distribute (outward):

- Variable $\rightarrow$ factor at the root: $\nu_{2b}(x_2) = \mu_{d2}(x_2) \mu_{e2}(x_2)$, $\nu_{2d}(x_2) = \mu_{b2}(x_2) \mu_{e2}(x_2)$, $\nu_{2e}(x_2) = \mu_{b2}(x_2) \mu_{d2}(x_2)$.
- Factor $\rightarrow$ variable: $\mu_{d1}(x_1) = \sum_{x_2} \psi(x_1, x_2) \nu_{2d}(x_2)$, $\mu_{e3}(x_3) = \sum_{x_2} \psi(x_2, x_3) \nu_{2e}(x_2)$.
- Variable $\rightarrow$ leaf factor: $\nu_{1a}(x_1) = \mu_{d1}(x_1)$, $\nu_{3c}(x_3) = \mu_{e3}(x_3)$.

Marginals: e.g. $p(x_2 \mid \bar{x}_E) \propto \nu_{2b}(x_2) \mu_{b2}(x_2) = \psi^E(x_2) \mu_{d2}(x_2) \mu_{e2}(x_2)$ and $p(x_1 \mid \bar{x}_E) \propto \nu_{1a}(x_1) \mu_{a1}(x_1) = \psi^E(x_1) \mu_{d1}(x_1)$. (Image source: M. I. Jordan, 2002.)

Proof: UGM Messages Equal Factor-Graph Messages

Claim: for a pairwise tree UGM converted to a factor graph (one factor $f_s = \psi(x_i, x_j)$ per edge, one singleton factor $\psi^E(x_j)$ per node), the UGM message $m_{ji}(x_i)$ equals the factor message $\mu_{si}(x_i)$.

UGM (Lecture 4): $m_{ji}(x_i) = \sum_{x_j} \left(\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}(x_j) \right).$

Factor graph: since $\mathcal{N}(s) = \{i, j\}$,

$$\begin{aligned}\mu_{si}(x_i) &= \sum_{x_{\mathcal{N}(s) \setminus i}} \left(f_s(x_{\mathcal{N}(s)}) \prod_{j \in \mathcal{N}(s) \setminus i} \nu_{js}(x_j) \right) = \sum_{x_j} \psi(x_i, x_j) \nu_{js}(x_j) \\ &= \sum_{x_j} \psi(x_i, x_j) \prod_{t \in \mathcal{N}'(j) \setminus s} \mu_{tj}(x_j) = \sum_{x_j} \left(\psi^E(x_j) \psi(x_i, x_j) \prod_{t \in \mathcal{N}'(j) \setminus s} \mu_{tj}(x_j) \right),\end{aligned}$$

where $\mathcal{N}'(j)$ is the neighborhood of X_j **omitting** the singleton factor node associated with $\psi^E(x_j)$ (whose message is $\mu = \psi^E(x_j)$). The remaining factors $t \in \mathcal{N}'(j) \setminus s$ correspond one-to-one with the UGM neighbors $k \in \mathcal{N}(j) \setminus i$, so by induction from the leaves $\mu_{tj}(x_j) = m_{kj}(x_j)$, and hence $\mu_{si}(x_i) = m_{ji}(x_i)$. ■

3 Maximum a Posteriori (MAP) Inference

Definition : Marginalization vs. MAP Problems

- **Marginalization problem:** **summing** over all configurations of sets of random variables.
- **Maximum a posteriori (MAP) problem:** **maximizing** over all sets of random variables. It has two aspects:
 1. finding the **maximal probability**, and
 2. finding a **configuration** that achieves the maximal probability.

Definition : Maximal Probability

Given $p(x \mid \bar{x}_E)$, where $\bar{X}_E$ is the set of observed variables, the maximum a posteriori probability is

$$\max_x p(x \mid \bar{x}_E) = \max_x \frac{p(x, \bar{x}_E)}{p(\bar{x}_E)} \longrightarrow \max_x p(x, \bar{x}_E) = \max_x p(x) \delta(x_E, \bar{x}_E) = \max_x p^E(x),$$

where $p^E(x)$ is the **unnormalized** representation of the conditional $p(x \mid \bar{x}_E)$. The denominator $p(\bar{x}_E)$ is a constant with respect to x , so it can be dropped.

Remark: dropping $p(\bar{x}_E)$ scales the maximal *value* by that constant but leaves the maximizing *configuration* unchanged; if the normalized value is needed, divide by $p(\bar{x}_E)$ (obtainable by sum-product).

Worked Example: The Fallacy of Maximizing Marginals Separately

Can we solve the MAP problem by (1) computing the marginal of **each variable** and (2) choosing for each variable the value that **maximizes its individual marginal**? **No!**
Consider $X \rightarrow Y$ with $x, y \in \{1, 2, 3\}$:

x	$p(x)$	$p(y x)$	$y=1$	$y=2$	$y=3$	$y=1$	$y=2$	$y=3$
1	.6	$x=1$	0	.6	.4	.4	.36	.24
2	.2	$x=2$	1	0	0			
3	.2	$x=3$	1	0	0			
						$p(y)$		

Individually, $\max_x p(x) = p(x=1) = 0.6$ and $\max_y p(y) = p(y=1) = 0.4$. But jointly,

$$\max_{x,y} p(x, y) = p(x=1, y=2) = 0.6 \times 0.6 = 0.36.$$

Scribe's note: the “marginal-wise” choice ($x=1, y=1$) actually has joint probability $p(x=1)p(y=1 | x=1) = 0.6 \times 0 = 0$ – it is not merely sub-optimal, it is **impossible**. (Image source: M. I. Jordan, 2002.)

3.1 From Marginal to MAP Algorithms

Key Rule: The Distributive Law Also Holds for max

The distributive law of multiplication over addition,

$$a \cdot b_1 + a \cdot b_2 + \dots + a \cdot b_n = a \cdot (b_1 + b_2 + \dots + b_n),$$

plays a key role in elimination and sum-product, e.g. with $a = p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)$,

$$\begin{aligned} p(x_1, \dots, x_5) &= \sum_{x_6} a \cdot p(x_6|x_2, x_5) = a \cdot p(x_6=0|x_2, x_5) + \dots + a \cdot p(x_6=k|x_2, x_5) \\ &= a \cdot (p(x_6=0|x_2, x_5) + \dots + p(x_6=k|x_2, x_5)) = a \sum_{x_6} p(x_6|x_2, x_5). \end{aligned}$$

The same law applies to the **max** operator (for $a \geq 0$):

$$\max(a \cdot b_1, a \cdot b_2, \dots, a \cdot b_n) = a \cdot \max(b_1, b_2, \dots, b_n).$$

So we can push “max” inward past factors that do not depend on the maximized variable:

$$\max_{x_6} p(x_1, \dots, x_6) = \underbrace{p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)}_{\text{independent of } x_6} \max_{x_6} p(x_6|x_2, x_5).$$

Replacing “sum” by “max” turns **elimination** into **MAP-elimination**, and **sum-product** into **max-product**.

Algorithm: MAP-Elimination (Jordan, 2002)

```

procedure MAP-ELIMINATE( $\mathcal{G}, E$ ) ▷ main steps
  INITIALIZE( $\mathcal{G}$ )
  EVIDENCE( $E$ )
  UPDATE( $\mathcal{G}$ )
  MAXIMUM
end procedure

```

```

procedure INITIALIZE( $\mathcal{G}$ )                                ▷ 1: same as variable elimination
  choose an ordering  $I$ 
  for each node  $X_i$  in  $\mathcal{V}$  do
    place  $p(x_i \mid x_{\pi_i})$  on the active list
  end for
end procedure

procedure EVIDENCE( $E$ )                                    ▷ 2: same as variable elimination
  for each  $i$  in  $E$  do
    place  $\delta(x_i, \bar{x}_i)$  on the active list
  end for
end procedure

procedure UPDATE( $\mathcal{G}$ )                                     ▷ 3: maximization; update active list
  for each  $i$  in  $I$  do
    find all potentials on the active list that reference  $x_i$  and remove them
    let  $\phi_i^{\max}(x_{T_i})$  denote the product of these potentials
    let  $m_i^{\max}(x_{S_i}) = \max_{x_i} \phi_i^{\max}(x_{T_i})$                                 ▷ sum  $\rightarrow$  max
    place  $m_i^{\max}(x_{S_i})$  on the active list
  end for
end procedure

procedure MAXIMUM                                         ▷ 4
   $\max_x p^E(x)$  = the scalar value on the active list
end procedure

```

Worked Example: MAP-Elimination on the Six-Node DGM

Using the running DGM of Lecture 4, $p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)p(x_6|x_2, x_5)$, evidence $\bar{X}_6$ and ordering $I = \{6, 5, 4, 3, 2, 1\}$:

$$\begin{aligned}
 \max_x p(x_1, \dots, x_5 \mid \bar{x}_6) &= \max_{x_1} \max_{x_2} \max_{x_3} \max_{x_4} \max_{x_5} \max_{x_6} \frac{p(x_1, x_2, x_3, x_4, x_5, \bar{x}_6)}{p(\bar{x}_6)} \\
 &\rightarrow \max_{x_1} \cdots \max_{x_6} p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)p(x_6|x_2, x_5) \delta(x_6, \bar{x}_6) \\
 &= \max_{x_1} p(x_1) \max_{x_2} p(x_2|x_1) \max_{x_3} p(x_3|x_1) \\
 &\quad \max_{x_4} p(x_4|x_2) \max_{x_5} p(x_5|x_3) \max_{x_6} p(x_6|x_2, x_5) \delta(x_6, \bar{x}_6),
 \end{aligned}$$

(dropping the constant $1/p(\bar{x}_6)$). Evaluating from the inside out defines the intermediate factors

$$\begin{aligned}
 m_6^{\max}(x_2, x_5) &= \max_{x_6} p(x_6|x_2, x_5) \delta(x_6, \bar{x}_6), & m_5^{\max}(x_2, x_3) &= \max_{x_5} p(x_5|x_3) m_6^{\max}(x_2, x_5), \\
 m_4^{\max}(x_2, x_3) &= \max_{x_4} p(x_4|x_2) m_5^{\max}(x_2, x_3), & m_3^{\max}(x_1, x_2) &= \max_{x_3} p(x_3|x_1) m_4^{\max}(x_2, x_3), \\
 m_2^{\max}(x_1) &= \max_{x_2} p(x_2|x_1) m_3^{\max}(x_1, x_2), & m_1^{\max} &= \max_{x_1} p(x_1) m_2^{\max}(x_1),
 \end{aligned}$$

and $m_1^{\max}$ is the MAP value $\max_x p^E(x)$. (Here $m_4^{\max}$ keeps the argument (x_2, x_3) because the

nested expression carries $m_5^{\max}(x_2, x_3)$ inside it; the slides label the final scalar as $m_1^{\max}(x_1)$. (Image source: M. I. Jordan, 2002.)

Worked Example: Maximal Probability Tables

Let $x_i \in \{0, 1\}$ and observe $\bar{X}_6 = 1$. As in Lecture 4, evidence takes a **2D slice** of the 3D table: with $p(x_6|x_2, x_5)$ entries $v_0, \dots, v_7$ (row order $x_2x_5x_6 = 000, \dots, 111$),

X_2	X_5	$m_6^{\max}(x_2, x_5) = \max_{x_6} p(x_6 x_2, x_5)\delta(x_6, \bar{x}_6)$
0	0	v_1
0	1	v_3
1	0	v_5
1	1	v_7

With $p(x_5|x_3)$ entries $b_1, \dots, b_4$ (row order $x_3x_5 = 00, 01, 10, 11$), $m_5^{\max}(x_2, x_3) = \max_{x_5} p(x_5|x_3) m_6^{\max}(x_2, x_5)$ is

X_2	X_3	$m_5^{\max}(x_2, x_3)$ (writing $m_6^{\max}(x_2, x_5)$ with its arguments' values)
0	0	$\max(p(x_5=0 x_3=0) m_6^{\max}(0, 0), p(x_5=1 x_3=0) m_6^{\max}(0, 1)) = \max(b_1 v_1, b_2 v_3)$
0	1	$\max(p(x_5=0 x_3=1) m_6^{\max}(0, 0), p(x_5=1 x_3=1) m_6^{\max}(0, 1)) = \max(b_3 v_1, b_4 v_3)$
1	0	$\max(p(x_5=0 x_3=0) m_6^{\max}(1, 0), p(x_5=1 x_3=0) m_6^{\max}(1, 1)) = \max(b_1 v_5, b_2 v_7)$
1	1	$\max(p(x_5=0 x_3=1) m_6^{\max}(1, 0), p(x_5=1 x_3=1) m_6^{\max}(1, 1)) = \max(b_3 v_5, b_4 v_7)$

The only change from the marginalization table of Lecture 4 is that “+” has become “max”.

Key Rule: Underflow and the Max-Sum Algorithm

Products of many probabilities (numbers between 0 and 1) tend to **underflow**. Since log is **monotone**, we can instead maximize

$$\arg \max_x p^E(x) = \arg \max_x \log p^E(x),$$

and the distributive law still holds for (max, +):

$$\max(a + b_1, a + b_2, \dots, a + b_n) = a + \max(b_1, b_2, \dots, b_n).$$

This turns the **max-product** algorithm into the **max-sum** algorithm. (The slides write $\max_x p^E(x) = \max_x \log p^E(x)$; strictly it is the maximizer that is shared, while the maximal values are related by log.)

3.2 The Max-Product Algorithm for Trees

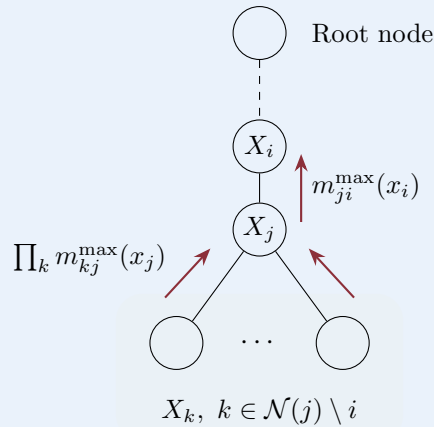
Definition : Max-Product Messages

To find the MAP probability for a tree, choose any node X_f as the **root** and propagate messages **inward** from the leaves. The message from X_j to X_i (closer to the root) is

$$m_{ji}^{\max}(x_i) = \max_{x_j} \left(\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}^{\max}(x_j) \right)$$

and the MAP probability is collected at the root:

$$\max_x p^E(x) = \max_{x_f} \left(\psi^E(x_f) \prod_{e \in \mathcal{N}(f)} m_{ef}^{\max}(x_f) \right).$$



Key Rule: No Outward Pass Is Needed for the MAP Value

Do we need to pass messages back to the leaves to get the MAP **probability**? **No!** The MAP probability is the same for every choice of root, so a single inward pass suffices.

Definition : MAP Configuration

The problem of finding a configuration x^* such that

$$x^* \in \arg \max_x p^E(x).$$

Using the messages to the root X_f , we obtain a value

$$x_f^* \in \arg \max_{x_f} \left(\psi^E(x_f) \prod_{e \in \mathcal{N}(f)} m_{ef}^{\max}(x_f) \right)$$

that necessarily belongs to **a** maximizing configuration.

Key Rule: Why a Plain Outward Pass Fails – and Back-Tracking

Could we run an outward max-product pass and pick $x_i^* \in \arg \max$ of each node's max-marginal independently? **No!** When several configurations tie for the maximum, there is **no guarantee** that the individually chosen values belong to the **same** maximizing configuration.

Solution: during the inward pass, also **record the maximizing values** in a table whenever

$m_{ji}^{\max}(x_i)$ is sent from X_j to X_i :

$$\delta_{ji}(x_i) \in \arg \max_{x_j} \left(\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}^{\max}(x_j) \right),$$

i.e. for each value of X_i , $\delta_{ji}(x_i)$ picks out a value of X_j (there can be several) that achieves the maximum. Then, in an **outward pass**:

1. choose a maximizing value x_f^* at the root X_f ;
2. set $x_e^* = \delta_{ef}(x_f^*)$ for each $e \in \mathcal{N}(f)$;
3. continue outward to the leaves.

Every choice is conditioned on its parent's already-fixed value, so the result is a **consistent** maximizing configuration (this is back-tracking, as in the Viterbi algorithm).

Notation warning: δ is overloaded in this course – $\delta(x_i, \bar{x}_i)$ is the evidence potential, $\delta_{ji}(x_i)$ is the back-pointer table above, and $\delta_{i \rightarrow j}$ (Section 4) is a junction-tree message.

Worked Example: Two Tied Paths in a Trellis

A **trellis (lattice)** diagram shows every possible state of each variable in a chain, with $x_n \in \{1, 2, 3\}$ (states $k = 1, 2, 3$). Two different paths (configurations) can give rise to the **same** MAP probability; choosing each x_n independently may then mix the two paths into a configuration that is **not** maximal. Back-pointers avoid this by always following one path. (Image source: C. Bishop.)

Algorithm: Max-Product for a Tree $\mathcal{T}(\mathcal{V}, \mathcal{E})$ (Jordan, 2002)

```

procedure MAX-PRODUCT( $\mathcal{T}, E$ )                                ▷ main steps
  EVIDENCE( $E$ )
   $f \leftarrow \text{CHOOSEROOT}(\mathcal{V})$ 
  for  $e \in \mathcal{N}(f)$  do
    COLLECT( $f, e$ )                                              ▷ 1
  end for
   $MAP = \max_{x_f} (\psi^E(x_f) \prod_{e \in \mathcal{N}(f)} m_{ef}^{\max}(x_f))$           ▷ MAP probability at root
   $x_f^* = \arg \max_{x_f} (\psi^E(x_f) \prod_{e \in \mathcal{N}(f)} m_{ef}^{\max}(x_f))$     ▷ MAP configuration at root
  for  $e \in \mathcal{N}(f)$  do
    DISTRIBUTE( $f, e$ )                                           ▷ 2
  end for
end procedure

procedure COLLECT( $i, j$ )                                       ▷ 1: inward message passing
  for  $k \in \mathcal{N}(j) \setminus i$  do
    COLLECT( $j, k$ )
  end for
  SENDMESSAGE( $j, i$ )
end procedure

procedure DISTRIBUTE( $i, j$ )                                     ▷ 2: outward message passing

```

```

SETVALUE( $i, j$ )
  for  $k \in \mathcal{N}(j) \setminus i$  do
    DISTRIBUTE( $j, k$ )
  end for
end procedure

procedure SENDMESSAGE( $j, i$ )
   $m_{ji}^{\max}(x_i) = \max_{x_j} (\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}^{\max}(x_j))$  ▷ MAP message
   $\delta_{ji}(x_i) \in \arg \max_{x_j} (\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}^{\max}(x_j))$  ▷ back-pointers
end procedure

procedure SETVALUE( $i, j$ ) ▷ MAP configuration in outward pass
   $x_j^* = \delta_{ji}(x_i^*)$ 
end procedure

```

Worked Example: Max-Product with Back-Tracking on a Chain

Find $\arg \max_{x_1, x_2, x_3} p(x_1, x_2, x_3)$ for the chain $X_1 - X_2 - X_3$ with $x_i \in \{0, 1\}$ and root X_2 .

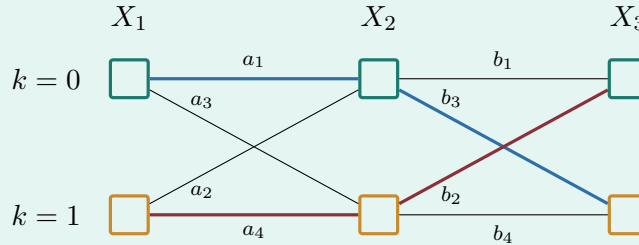
Inward pass. Write $a_1 = \psi(x_1=0)\psi(x_1=0, x_2=0)$, $a_2 = \psi(x_1=1)\psi(x_1=1, x_2=0)$, $a_3 = \psi(x_1=0)\psi(x_1=0, x_2=1)$, $a_4 = \psi(x_1=1)\psi(x_1=1, x_2=1)$, and analogously b_1, b_3 ($x_3 = 0, 1$ with $x_2 = 0$) and b_2, b_4 ($x_3 = 0, 1$ with $x_2 = 1$). Assume $a_1 > a_2$, $a_4 > a_3$, $b_3 > b_1$, $b_2 > b_4$.

X_2	$m_{12}^{\max}(x_2) = \max_{x_1} \psi(x_1) \psi(x_1, x_2)$	$\delta_{12}(x_2)$
0	$\max(a_1, a_2) = a_1$	$x_1^{\max} = 0$
1	$\max(a_3, a_4) = a_4$	$x_1^{\max} = 1$
X_2	$m_{32}^{\max}(x_2) = \max_{x_3} \psi(x_3) \psi(x_3, x_2)$	$\delta_{32}(x_2)$
0	$\max(b_1, b_3) = b_3$	$x_3^{\max} = 1$
1	$\max(b_2, b_4) = b_2$	$x_3^{\max} = 0$

At the root. $\max_{x_2} \psi(x_2) m_{12}^{\max}(x_2) m_{32}^{\max}(x_2) = \max(\psi(x_2=0) a_1 b_3, \psi(x_2=1) a_4 b_2) = \max(d_1, d_2)$. Assume a **tie** $d_1 = d_2$, so $x_2^{\max} \in \{0, 1\}$.

Outward (downward) pass using the back-pointers:

$$\begin{aligned}
 x_2^* = 0 : \quad & x_1^* = \delta_{12}(0) = 0, \quad x_3^* = \delta_{32}(0) = 1, \\
 x_2^* = 1 : \quad & x_1^* = \delta_{12}(1) = 1, \quad x_3^* = \delta_{32}(1) = 0.
 \end{aligned}$$



The two MAP configurations are $(0, 0, 1)$ and $(1, 1, 0)$. Picking values independently could, e.g., combine $x_1^* = 0$ (blue path) with $x_2^* = 1$ (red path), which lies on neither maximizing path – exactly the failure the back-pointers prevent.

4 The Junction Tree Algorithm

Key Rule: From Variable Elimination to Junction Trees

- Variable elimination is **query sensitive**: we must re-run the entire algorithm for each query node. The **junction tree algorithm** generalizes variable elimination to avoid this.
- **Main idea**: probability distributions corresponding to **loopy** undirected graphs can be **re-parameterized as trees** (of clusters of variables), and we can then run the **sum-product** algorithm on this tree re-parameterization.

4.1 Cluster Graphs

Definition : Cluster Graph

An undirected graph such that:

1. **nodes** are **clusters** $C_i \subseteq \{X_1, \dots, X_n\}$ of random variables;
2. each **edge** between C_i and C_j is associated with a **sepset** $S_{ij} \subseteq C_i \cap C_j$ (in the junction trees built below, $S_{ij} = C_i \cap C_j$).

A cluster graph organizes overlapping variable clusters and records their shared variables through sepsets.

Definition : Family Preservation and Cluster Potentials

Family preservation. Let $\Psi = \{\psi_1, \dots, \psi_K\}$ be the potentials of the UGM, and define an **assignment function**

$$\alpha : \Psi \mapsto \{1, \dots, m\},$$

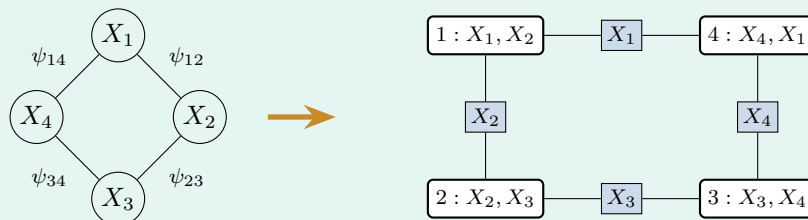
where $\alpha(\psi) = j$ means potential ψ is assigned to cluster C_j ; this requires $\text{Scope}[\psi] \subseteq C_{\alpha(\psi)}$.

Cluster potential. For cluster C_j , define

$$\phi_j = \prod_{\psi: \alpha(\psi)=j} \psi, \quad \text{so that} \quad \prod_{\psi \in \Psi} \psi = \prod_j \phi_j.$$

Each potential is assigned to **exactly one** cluster, i.e. every ψ is used **once**.

Worked Example: Cluster Graph for a Four-Cycle



Cluster potentials: $\phi_1(x_1, x_2) = \psi_{12}(x_1, x_2)$, $\phi_2(x_2, x_3) = \psi_{23}(x_2, x_3)$, $\phi_3(x_3, x_4) = \psi_{34}(x_3, x_4)$, $\phi_4(x_1, x_4) = \psi_{14}(x_1, x_4)$. Note this cluster graph itself contains a **loop**. (Adapted from D. Koller, “Probabilistic Graphical Models”.)

4.2 The Running Intersection Property

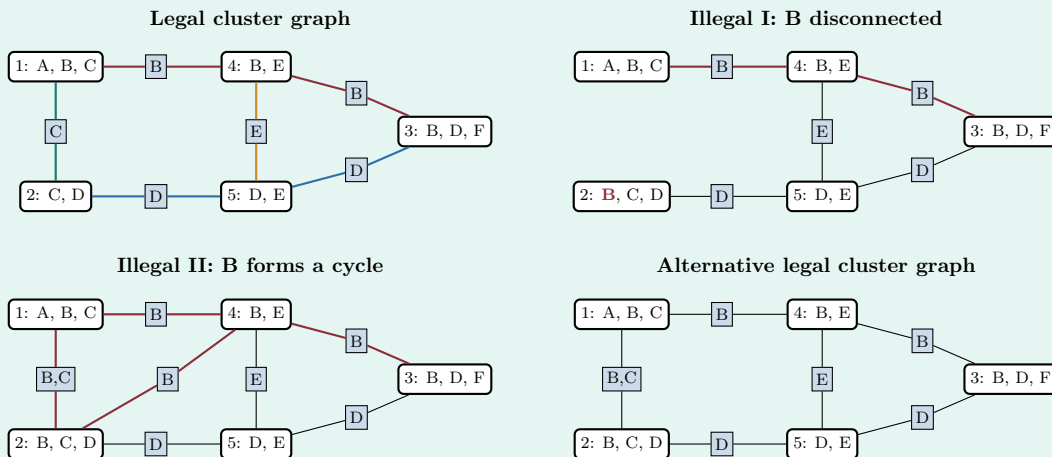
Definition : Running Intersection Property (Junction Tree Property)

For each pair of clusters C_i, C_j and each variable $X \in C_i \cap C_j$, there exists a **unique path** between C_i and C_j for which **all clusters and sepsets contain X** .

Equivalently: for any X , the set of clusters and sepsets containing X **forms a tree**. A valid cluster graph **must** fulfil this property.

Worked Example: Legal and Illegal Cluster Graphs

Each picture shows clusters (white) and sepsets (blue). Coloured edges trace the sub-graph of a variable: **B**, **C**, **D**, **E**.



- **Legal:** every variable's clusters and sepsets form a tree, e.g. B: 1 – 4 – 3; D: 2 – 5 – 3.
- **Illegal I:** cluster 2 now contains B, but no B-carrying sepset reaches it – **B is disconnected** from the path 1 – 4 – 3.
- **Illegal II:** adding sepsets $\{B, C\}$ (1–2) and $\{B\}$ (2–4) connects B, but the B-sub-graph 1 – 2 – 4 – 1 **forms a cycle**.
- **Alternative legal:** with sepset $\{B, C\}$ on 1–2 and **no** 2–4 edge, B's sub-graph is the tree 2 – 1 – 4 – 3.

Note that a valid cluster graph may itself contain loops (all four graphs do); the property constrains only each variable's sub-graph. (Adapted from D. Koller.)

Definition : Cluster Tree and Clique Tree (Junction Tree)

- A cluster graph **without cycles** is a **cluster tree**.
 - A cluster tree that fulfils the **running intersection property** is a **clique tree**, a.k.a. **junction tree**.
 - In a clique tree, we call a “cluster” a **clique** and a “cluster potential” a **clique potential**.
- We first see how to compute all marginals on a junction tree, and then how to **convert a DGM/UGM into a junction tree**.

4.3 Sum-Product on a Junction Tree

Key Rule: Junction Tree Sum-Product

Choose a **root clique** (arbitrarily), then pass **inward** messages from the leaf cliques to the root, and **outward** messages from the root back to the leaves.

Protocol: C_i is ready to send a message to a neighbor C_j when it has received messages from **all neighbors except** C_j .

Message from C_i to C_j – multiply the clique potential by all other incoming messages and sum out the variables **not** in the sepset:

$$\delta_{i \rightarrow j} = \sum_{C_i \setminus S_{ij}} \phi_i \cdot \prod_{k \in \mathcal{N}(i) \setminus j} \delta_{k \rightarrow i}$$

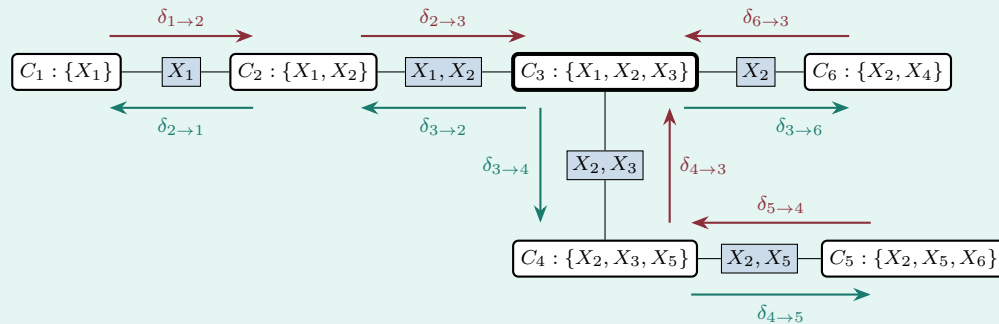
Unnormalized clique marginal:

$$\tilde{p}(C_i) = \phi_i \cdot \prod_{k \in \mathcal{N}(i)} \delta_{k \rightarrow i}$$

It is unnormalized because the clique potentials come from the UGM potentials, where the partition function was ignored. Normalize by summing $\tilde{p}(C_i)$ over C_i ; single-variable marginals follow by summing out the other clique members.

Worked Example: Junction Tree Messages with C_3 as Root

Cliques (with potentials) and sepsets: $C_1 : \{X_1\}$ (ϕ_1), $C_2 : \{X_1, X_2\}$ (ϕ_2), $C_3 : \{X_1, X_2, X_3\}$ (ϕ_3), $C_4 : \{X_2, X_3, X_5\}$ (ϕ_4), $C_5 : \{X_2, X_5, X_6\}$ (ϕ_5), $C_6 : \{X_2, X_4\}$ (ϕ_6).



Inward pass (red):

$$\delta_{1 \rightarrow 2} = \sum_{C_1 \setminus S_{12}} \phi_1 = \phi_1,$$

$$\delta_{5 \rightarrow 4} = \sum_{C_5 \setminus S_{45}} \phi_5 = \sum_{X_6} \phi_5,$$

$$\delta_{6 \rightarrow 3} = \sum_{C_6 \setminus S_{36}} \phi_6 = \sum_{X_4} \phi_6,$$

$$\delta_{2 \rightarrow 3} = \sum_{C_2 \setminus S_{23}} \phi_2 \cdot \delta_{1 \rightarrow 2} = \phi_2 \cdot \phi_1,$$

$$\delta_{4 \rightarrow 3} = \sum_{C_4 \setminus S_{34}} \phi_4 \cdot \delta_{5 \rightarrow 4} = \sum_{X_5} \phi_4 \sum_{X_6} \phi_5.$$

(Here $C_1 \setminus S_{12} = C_2 \setminus S_{23} = \emptyset$, so no summation is performed.)

Outward pass (teal):

$$\begin{aligned}\delta_{3 \rightarrow 2} &= \sum_{C_3 \setminus S_{23}} \phi_3 \cdot \delta_{6 \rightarrow 3} \cdot \delta_{4 \rightarrow 3}, & \delta_{3 \rightarrow 6} &= \sum_{C_3 \setminus S_{36}} \phi_3 \cdot \delta_{2 \rightarrow 3} \cdot \delta_{4 \rightarrow 3}, \\ \delta_{3 \rightarrow 4} &= \sum_{C_3 \setminus S_{34}} \phi_3 \cdot \delta_{2 \rightarrow 3} \cdot \delta_{6 \rightarrow 3}, & \delta_{2 \rightarrow 1} &= \sum_{C_2 \setminus S_{12}} \phi_2 \cdot \delta_{3 \rightarrow 2}, \\ \delta_{4 \rightarrow 5} &= \sum_{C_4 \setminus S_{45}} \phi_4 \cdot \delta_{3 \rightarrow 4}.\end{aligned}$$

Clique marginals:

$$\begin{aligned}\tilde{p}(C_1) &= \tilde{p}(X_1) = \phi_1 \cdot \delta_{2 \rightarrow 1} = \phi_1 \cdot \sum_{C_2 \setminus S_{12}} \phi_2 \cdot \delta_{3 \rightarrow 2} = \phi_1 \cdot \sum_{X_2} \phi_2 \cdot \sum_{C_3 \setminus S_{23}} \phi_3 \cdot \delta_{6 \rightarrow 3} \cdot \delta_{4 \rightarrow 3} \\ &= \phi_1 \cdot \sum_{X_2} \phi_2 \cdot \sum_{X_3} \phi_3 \cdot \sum_{X_4} \phi_4 \cdot \sum_{X_5} \phi_5 \cdot \sum_{X_6} \phi_6, & p(X_1) &= \frac{\tilde{p}(X_1)}{\sum_{X_1} \tilde{p}(X_1)}.\end{aligned}$$

This result is equivalent to variable elimination! Similarly,

$$\begin{aligned}\tilde{p}(C_2) &= \tilde{p}(X_1, X_2) = \phi_2 \cdot \delta_{1 \rightarrow 2} \cdot \delta_{3 \rightarrow 2}, \\ p(X_1, X_2) &= \frac{\tilde{p}(X_1, X_2)}{\sum_{X_1} \sum_{X_2} \tilde{p}(X_1, X_2)}, & p(X_2) &= \sum_{X_1} p(X_1, X_2), \\ \tilde{p}(C_3) &= \tilde{p}(X_1, X_2, X_3) = \phi_3 \cdot \delta_{2 \rightarrow 3} \cdot \delta_{6 \rightarrow 3} \cdot \delta_{4 \rightarrow 3}, \\ \tilde{p}(C_4) &= \tilde{p}(X_2, X_3, X_5) = \phi_4 \cdot \delta_{3 \rightarrow 4} \cdot \delta_{5 \rightarrow 4}, \\ \tilde{p}(C_5) &= \tilde{p}(X_2, X_5, X_6) = \phi_5 \cdot \delta_{4 \rightarrow 5}, \\ \tilde{p}(C_6) &= \tilde{p}(X_2, X_4) = \phi_6 \cdot \delta_{3 \rightarrow 6}.\end{aligned}$$

(The slide writes $\delta_{4 \rightarrow 3} = \sum \phi_4 \cdot \delta_{5 \rightarrow 3}$; the incoming message is $\delta_{5 \rightarrow 4}$.)

4.4 Constructing the Junction Tree

Key Rule: Three Steps to a Junction Tree

1. **Triangulation:** choose an elimination ordering I and get the **reconstituted graph** (moralize first if the model is a DGM).
2. **Clusters and sepsets:** use the **elimination cliques** as clusters; every pair with non-empty overlap gives a candidate sepset $S_{ij} = C_i \cap C_j$. **Assign cluster potentials** from the conditional probabilities (DGM) or potentials (UGM), using each one **exactly once**.
3. **Clique tree:** find the **maximum spanning tree** over the clusters, with the **cardinality of each sepset** as the edge weight.

Algorithm: Step 1 – Triangulation (Jordan, 2002)

```

procedure DIRECTEDGRAPHELIMINATE( $\mathcal{G}, I$ )
   $\mathcal{G}^m \leftarrow \text{MORALIZE}(\mathcal{G})$                                  $\triangleright$  for DGM; skip this step if UGM
  UNDIRECTEDGRAPHELIMINATE( $\mathcal{G}^m, I$ )                         $\triangleright$  get reconstituted graph
end procedure

procedure MORALIZE( $\mathcal{G}$ )
  for each node  $X_i$  in  $I$  do
    connect all of the parents of  $X_i$ 

```

```

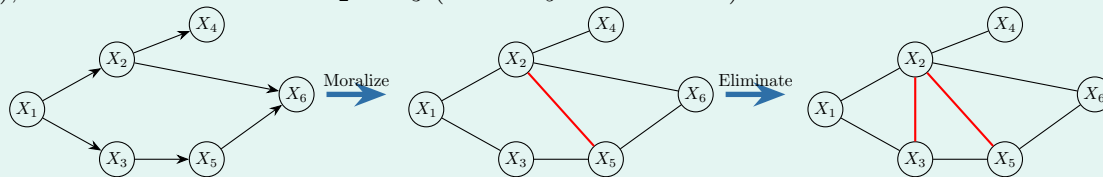
end for
  drop the orientation of all edges
  return  $\mathcal{G}$ 
end procedure

procedure UNDIRECTEDGRAPHELIMINATE( $\mathcal{G}, I$ )
  for each node  $X_i$  in  $I$  do
    connect all of the remaining neighbors of  $X_i$ 
    remove  $X_i$  from the graph
  end for
end procedure

```

Worked Example: Building the Junction Tree of the Six-Node DGM

Step 1 (triangulation) with $I = (6; 5; 4; 3; 2; 1)$: moralization marries X_2 and X_5 (parents of X_6); elimination then adds $X_2 - X_3$ (when X_5 is eliminated).



Step 2 (clusters, sepsets and potentials). The elimination cliques give the clusters (labelled as on the slides)

$$\begin{aligned}
 C_1 &: \{X_1\}, & C_2 &: \{X_1, X_2\}, & C_3 &: \{X_1, X_2, X_3\}, \\
 C_4 &: \{X_2, X_3, X_5\}, & C_5 &: \{X_2, X_5, X_6\}, & C_6 &: \{X_2, X_4\}.
 \end{aligned}$$

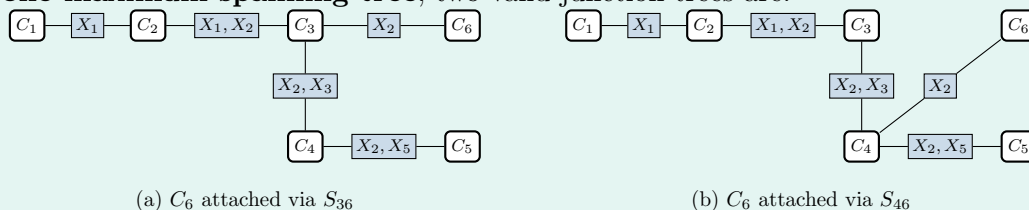
All candidate sepsets and their weights $|S_{ij}|$:

Sepset	Variables	$ S_{ij} $	Sepset	Variables	$ S_{ij} $	Sepset	Variables	$ S_{ij} $
S_{12}	X_1	1	S_{24}	X_2	1	S_{35}	X_2	1
S_{13}	X_1	1	S_{25}	X_2	1	S_{36}	X_2	1
S_{23}	X_1, X_2	2	S_{26}	X_2	1	S_{45}	X_2, X_5	2
S_{34}	X_2, X_3	2	S_{46}	X_2	1	S_{56}	X_2	1

Assigning each conditional probability of $p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)p(x_6|x_2, x_5)$ **once** (for a UGM, use $\psi(x_1)\psi(x_1, x_2)\psi(x_1, x_3)\psi(x_2, x_4)\psi(x_3, x_5)\psi(x_2, x_5, x_6)$ in the same way):

$$\begin{aligned}
 \phi_1(X_1) &= p(x_1), & \phi_2(X_1, X_2) &= p(x_2|x_1), & \phi_3(X_1, X_2, X_3) &= p(x_3|x_1), \\
 \phi_4(X_2, X_3, X_5) &= p(x_5|x_3), & \phi_5(X_2, X_5, X_6) &= p(x_6|x_2, x_5), & \phi_6(X_2, X_4) &= p(x_4|x_2).
 \end{aligned}$$

Step 3 (maximum spanning tree). Kruskal first takes the three weight-2 edges S_{23}, S_{34}, S_{45} (linking $C_2 - C_3 - C_4 - C_5$), then weight-1 edges to attach C_1 and C_6 . **There can be more than one maximum spanning tree**; two valid junction trees are:



Tree (a) is the junction tree used in the message-passing example above. Both have total weight $1 + 2 + 2 + 2 + 1 = 8$.

Scribe's check: the upper bound in the theorem below is $\sum_i |C_i| - N = (1+2+3+3+3+2) - 6 = 8$, which both trees attain – consistent with both satisfying the running intersection property (X_2 appears in $C_2, \dots, C_6$, which are connected through X_2 -carrying sepsets in either tree). (Image source: M. I. Jordan, 2002.)

Algorithm: Kruskal's Maximum Spanning Tree

```

procedure KRUSKAL( $G$ )
   $A \leftarrow \emptyset$ 
  for each  $v \in G.V$  do
    MAKE-SET( $v$ )
  end for
  for each  $(u, v)$  in  $G.E$  ordered by weight( $u, v$ ), decreasing do
    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ) then
       $A \leftarrow A \cup \{(u, v)\}$ 
      UNION( $u, v$ )
    end if
  end for
  return  $A$ 
end procedure

```

(Source: Wikipedia, “Kruskal’s algorithm”, with the edge order reversed – **decreasing** weight – to obtain a **maximum** spanning tree.)

Key Rule: Junction Trees Are Maximum Spanning Trees

Theorem: a cluster tree T is a clique tree / junction tree **only if** it is a **maximal spanning tree** (with sepset cardinalities as edge weights).

Proof: Junction Tree $\Rightarrow$ Maximum Spanning Tree

Let T be a cluster tree with M clusters C_i and $M - 1$ sepsets S_j , over N random variables. Let $\mathbf{1}(a)$ be the indicator function (1 if a is true, 0 otherwise).

Step 1 (one variable). Fix a variable X_k . Every sepset containing X_k joins two clusters that both contain X_k , so these sepsets are edges of a sub-forest on the clusters containing X_k . A forest has at most $\#nodes - 1$ edges (“minus 1” because a tree has $\#edges = \#nodes - 1$):

$$\underbrace{\sum_{j=1}^{M-1} \mathbf{1}(X_k \in S_j)}_{\# \text{ times } X_k \text{ appears in the sepsets}} \leq \underbrace{\sum_{i=1}^M \mathbf{1}(X_k \in C_i)}_{\# \text{ times } X_k \text{ appears in the clusters}} - 1,$$

with **equality** exactly when X_k ’s clusters and sepsets form a single sub-tree, i.e. when the **running intersection property** holds for X_k .

Step 2 (total weight). The weight of T is the sum of its sepset cardinalities:

$$\begin{aligned}
 w(T) &= \sum_{j=1}^{M-1} |S_j| = \sum_{j=1}^{M-1} \sum_{k=1}^N \mathbf{1}(X_k \in S_j) = \sum_{k=1}^N \sum_{j=1}^{M-1} \mathbf{1}(X_k \in S_j) && \text{sum of cardinalities of all sepsets} \\
 &\leq \sum_{k=1}^N \left[\sum_{i=1}^M \mathbf{1}(X_k \in C_i) - 1 \right] && \text{from Step 1} \\
 &= \sum_{i=1}^M \sum_{k=1}^N \mathbf{1}(X_k \in C_i) - N = \sum_{i=1}^M |C_i| - N. && \text{cluster sizes minus } N
 \end{aligned}$$

Step 3. For a junction tree the running intersection property holds for **every** X_k , so every inequality in Step 1 is an equality and $w(T)$ attains the upper bound $\sum_i |C_i| - N$. No cluster tree can exceed this bound, so T has the **maximum** possible sum of sepset cardinalities, i.e. it is a **maximal spanning tree**. ■

Scribe's note (converse in practice): when the clusters are the cliques of a triangulated graph, a junction tree is known to exist, so the maximum weight **equals** the bound. Any maximum spanning tree then also attains the bound, which forces equality for every X_k – hence every maximum spanning tree found by Kruskal satisfies the running intersection property.

Summary: Three Message-Passing Algorithms

	Factor-tree sum-product	Max-product	Junction tree
Graph	factor tree (incl. polytrees)	tree (or factor tree)	any DGM/UGM, after triangulation
Node type	variables and factors	variables	cliques (clusters)
Message	ν_{is}, μ_{si} (sum)	$m_{ji}^{\max}$ (max) and δ_{ji}	$\delta_{i \rightarrow j}$ (sum over $C_i \setminus S_{ij}$)
Passes	collect + distribute	collect; outward only for back-tracking	collect + distribute
Output	all singleton marginals	MAP value and a MAP configuration	all clique (hence singleton) marginals

Take-away: all three are the distributive law applied on a tree; they differ only in **what** the tree's nodes are and **which** operator ($\sum$ or $\max$) is pushed inward.

5 Summary

- Represented a joint distribution with a factor graph, and used it to compute the marginal/conditional probabilities.
- Used the max-product algorithm to find the maximal probability and its configurations.
- Converted a DGM/UGM into the junction tree and used it to compute the marginal/conditional probabilities.

Key Takeaways

1. A **factor graph** is a bipartite graph of variable and factor nodes that makes the factorization $p(\mathbf{x}) = \prod_s f_s(\mathbf{x}_s)$ **explicit**. DGMs (one factor per family) and UGMs (one factor per maximal clique) convert directly, and **local cycles** – e.g. in moralized polytrees – can

disappear, leaving a tree.

2. **Sum-product on factor trees** uses $\nu_{is}(x_i) = \prod_{t \in \mathcal{N}(i) \setminus s} \mu_{ti}(x_i)$ and $\mu_{si}(x_i) = \sum_{x_{\mathcal{N}(s) \setminus i}} f_s \prod_{j \in \mathcal{N}(s) \setminus i} \nu_{js}$, with $p(x_i) \propto \nu_{is} \mu_{si}$; on pairwise UGMs, μ_{si} coincides with the Lecture 4 message m_{ji} .
3. **MAP** is **not** solved by maximizing each marginal separately. Because the distributive law also holds for max (and for $(\max, +)$ in the log domain, avoiding underflow), replacing $\sum$ by max gives **MAP-elimination** and **max-product**.
4. The MAP **value** needs only an inward pass; a consistent MAP **configuration** needs back-pointers $\delta_{ji}(x_i)$ recorded on the way in and followed on the way out, because ties can otherwise mix different optimal paths.
5. A **junction (clique) tree** is a cluster tree satisfying the **running intersection property**; with family-preserving clique potentials, sum-product with messages $\delta_{i \rightarrow j} = \sum_{C_i \setminus S_{ij}} \phi_i \prod_{k \in \mathcal{N}(i) \setminus j} \delta_{k \rightarrow i}$ yields all clique marginals $\tilde{p}(C_i) = \phi_i \prod_{k \in \mathcal{N}(i)} \delta_{k \rightarrow i}$ – equivalent to variable elimination, but for all queries at once.
6. To **build** a junction tree: moralize (DGM), triangulate via an elimination ordering, take the elimination cliques as clusters, assign each potential once, and take a **maximum spanning tree** weighted by sepset cardinality (e.g. Kruskal). A junction tree must be a maximum spanning tree, since only then does $\sum_j |S_j|$ reach $\sum_i |C_i| - N$.

Acknowledgement: Slide content adapted from M. I. Jordan, “An Introduction to Probabilistic Graphical Models” (2002, Chapters 4.2, 4.3 and 17); D. Koller & N. Friedman, “Probabilistic Graphical Models” (Ch. 10); D. Barber, “Bayesian Reasoning and Machine Learning” (Ch. 6); K. Murphy, “Machine Learning: A Probabilistic Perspective” (Ch. 20.4); C. Bishop, “Pattern Recognition and Machine Learning” (Ch. 8.4.3).